

Codexa OS Knowledge Graph: A Token-Reduction Proof for LLM Code Retrieval

Rohit V K | CODE & FULL WRITE-UP: <https://github.com/rohitvijayakumar45/Codexa> | <https://github.com/rohitvijayakumar45>

85.8x	98.8%	174	5
fewer context tokens	token reduction	code symbols tested	real repositories

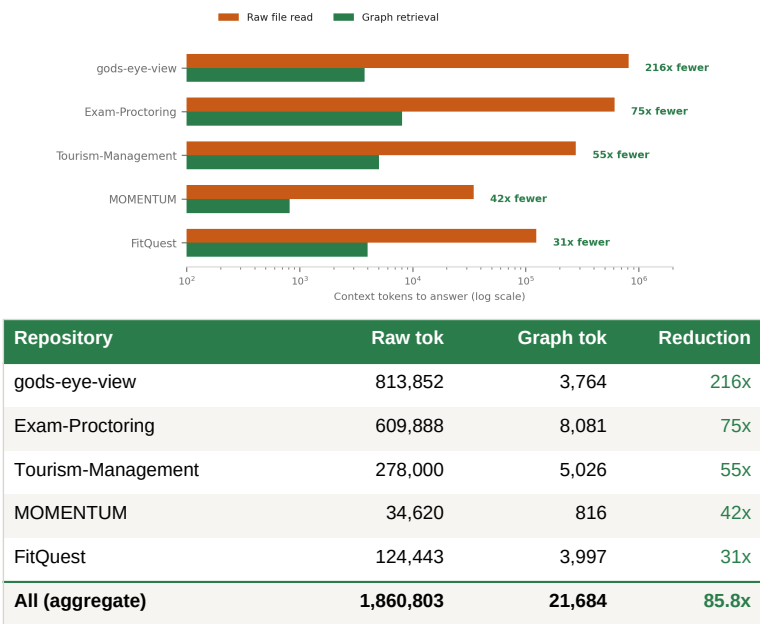
PROBLEM STATEMENT

LLM coding agents that answer from raw source files suffer four compounding failures: **hallucination** (no ground truth to check a claim against), **context bloat** (stale files pile up in the window), **runaway token usage** (cost scales with file size and reference count — hard questions can make an agent fail to converge at all; single runs we tested reached 300K–1.9M tokens with no answer), and **sycophancy** (a confidently-worded wrong answer talks a correct peer agent out of its own). Codexa's structural graph and persistent memory address all four: claims are checked against the graph instead of self-reported (hallucination), stale payloads are evicted by graph distance instead of a flat timer (bloat), the graph replaces raw-file reading (token usage — measured below), and disagreement is resolved by verifying claims against the graph before any free-text debate (sycophancy).

METHODOLOGY

Pipeline: each repository is parsed with real **tree-sitter** grammars (Python, JavaScript, TypeScript, TSX) into an incremental concrete syntax tree, from which an **event-sourced structural knowledge graph** is built — nodes for files, functions and classes; edges for imports, calls and dependencies — derived from the actual parse tree, not regex or text pattern-matching. An LLM symbol-annotation pass then walks that graph once and writes a one-line semantic description of what each symbol does, cached by a content hash of its source span so a later re-ingest only re-annotates symbols whose code actually changed. Those annotations, plus three other memory types (Novelty — four in total: semantic, episodic, procedural, organizational), persist in a shared project memory store used by every model regardless of which LLM is active. **Measurement:** a deterministic, model-free comparison isolates the graph as the only variable, with no LLM in the loop. Per symbol, *graph retrieval* resolves its definition and every caller/dependent directly from the graph in one lookup; *raw retrieval* opens every file needed to derive the same facts, with that file set itself read from the graph's own incoming edges, so both arms answer the identical question. Tokens are counted with tiktoken (cl100k_base). An earlier end-to-end agentic benchmark across 3 LLMs was abandoned once every arm was found to thrash without converging on hard multi-hop questions — a convergence failure that would have measured model instability, not the architecture, and a failure mode structurally related to the sycophancy problem Quorum solves (Novelty).

RESULTS & INSIGHTS



At an illustrative **\$0.25/1M input tokens**, these 174 lookups cost **\$0.465 raw** vs. **\$0.005 via the graph** — scaled to 100K lookups, roughly **\$267 vs. \$3**, a direct saving at any provider's price.

Key insight: this raw baseline is conservative — real agentic runs on the same questions re-read files and chase leads, reaching 300K–1.9M tokens *without even converging*, so the true gap is larger than shown. Additive saving: Codexa's memory injects a fixed 0.2K–18K-token payload answering conventions without re-deriving them.

NOVELTY

- Four memory types:** semantic, episodic, procedural, organizational — durable, shared across every model regardless of which LLM is active.
- Symbol annotations:** an LLM writes what each symbol MEANS on top of what tree-sitter's parse already captures about its structure, cached by content hash so restarts reuse it for free instead of re-annotating unchanged code.
- Graph:** an event-sourced structural knowledge graph (files/functions/classes as nodes, imports/calls/dependencies as edges) that answers navigation queries directly instead of re-reading files, and doubles as the mechanism that prevents context bloat — stale tool payloads are evicted by graph-distance from the file in play, not a fixed round-count.
- Blast radius:** a reverse-dependency traversal of the same graph, run from any changed symbol, that surfaces every downstream node reachable by import/call/dependency edges — impact analysis before the change is made, not after.
- Quorum:** agents answer independently, checked against the graph before seeing a peer's answer; ties debate only via structured belief cards, revising solely on a verified claim — closing the sycophancy vector.